



GTK+ Programming: Glade, GLib and GCC

In this part of our tutorial series, we will adapt the application we developed earlier to implement the control code using GLib and GCC. We will use the same GUI front-end we developed in Glade for the employee application, to show how easy it is to switch the underlying language to implement control logic. At the same time, we will start using some of the advanced features of GTK+ as well.

GLib is a general-purpose utility library released under the GNU LGPL, which provides many useful data types, macros, type conversions, string utilities, file utilities and so on. GLib is ported to Linux, Windows, OS/2 and many other OSs. The latest stable version of GLib is 2.24. Using GLib-supplied data types makes it easier to write portable code for UI programming. Let us use some data types and utility functions provided by GLib, especially the string utility functions. For example, to create a linked list of radio buttons, we can use *GSList*, a singly-linked list implementation. Other useful data types provided include *gchar* (equivalent to the *char* data type in C), *gint*, etc. Please check the box, 'An introduction to GLib and data types' for a brief explanation on the subject.

The structure of C code

Since we already have a good idea about the application and the GUI created using Glade, let's start implementing the control code. The source files are split into:

- A header file containing custom macros, which includes private data structures, handler function declarations, etc
- A C source file containing function definitions
- A makefile with rules for building the application
- The private data structure *struct empdata{...}* contains pointers to all the widgets that we need to access at runtime. You might note that instead of using their specific types (like *GtkTextEntry* for the text entry widget), we have used *GtkWidget* to hold these pointers. Please check the box titled, 'Typecasting macros' for more details.

The main() function

Invoke *gtk_init()* early during execution, using the parameters passed to *main()*. This enables any command-line parameter processing specific to GTK+ applications (like *-geometry*)

to take effect. *gtk_init()* consumes any parameters specific to GTK+, and leaves the other parameters intact for the application to process. Please note that to facilitate this, you need to pass pointers to the *argv* and *argc* parameters.

In the main function, allocate your private data structure (an instance of *struct empdata*) using a GLib utility function *g_malloc()*. As the name implies, this function allocates memory of the requested size, and fills it with zeros.

Next, let us create an instance of a *GtkBuilder* object, using *gtk_builder_new()*, to parse the *GtkBuilder* format file for our GUI, and create the appropriate widgets. Let's add the resource file (defined in the header file) using the method *gtk_builder_add_from_file()*. There are other ways of adding *GtkBuilder* files into the object. For example, if you don't want a separate *GtkBuilder* file as a resource, you can have it stored in a string or as a header file, and have it compiled into the binary. Please check the GTK+ documentation for further details on this. Once you add the file to the *GtkBuilder* instance, you can populate your private data structure with widgets by using your own macro, *GET_OBJ_FROM_BUILDER()*, which is defined in *employee.h*.

It's time to connect all the signal handlers with the instance of the private data structure. Let's use the simple function *gtk_builder_connect_signals()*. This opens and checks the application's symbol table, and tries to match the handler name to the interface description in the *GtkBuilder* file. The *user_data* to this function (in our case, the private data structure) becomes the *user_data* to all signal functions. A more fine-grained variant of the connect function is available (*gtk_builder_connect_signals_full()*) if you want to individually connect signal handlers with different *user_data* pointers.

After connecting the signals, let's build a context ID for the status bar, using *gtk_statusbar_get_context_id()* and a seed string. As you may remember, any string is good, as long as it can identify the context uniquely. Next, the notebook widget